Git & GitHub
Practical Workbook

Modules 2 and 3

Clone • Branch • Code • Commit • Push • Pull Request • Merge Conflict

In this workbook, you will work with an existing calculator project, make a small change on your own branch, publish your work to GitHub, and practise resolving a merge conflict.

Repository: https://github.com/Vikashraise/calculator

This repository is private. Make sure you have access before cloning. If you cannot push to the original repository, use an authorized fork or your own copy.

1. Git and GitHub: the essentials

Git vs GitHub

Git tracks changes to files and helps you work with versions, branches, and merges. **GitHub** hosts Git repositories and supports collaboration through pull requests, reviews, and issues.

The key terms

Term	Meaning
Repository	The project and its tracked history.
Clone	Create a local copy of a repository, including its Git history.
Remote	A named connection to another repository.
origin	The conventional remote name created when you clone a repository.
Branch	A separate line of development for a feature or fix.
main	The commonly used default branch.
Working directory	The files you are currently viewing and editing.
Staging area	The changes selected for your next commit.
Commit	A saved snapshot in local Git history.
Push	Upload your local commits to a remote repository.
Pull request (PR)	A proposal to review and merge changes from one branch into another.
Merge conflict	A situation where Git cannot automatically combine competing edits.

Remember: a commit is saved locally. A push uploads commits to GitHub. Creating a pull request proposes a merge; it does not mean the changes are already merged.

2. Clone the calculator repository

Step 1 — Copy the repository URL

On GitHub, open the calculator repository, click **Code**, choose **HTTPS**, and copy the URL:

```
https://github.com/Vikashraise/calculator.git
```

Step 2 — Open PowerShell

Choose a folder where you keep your projects. For example:

```
cd $HOME\Projects
```

If the Projects folder does not exist yet, create it:

```
New-Item -ItemType Directory -Force "$HOME\Projects"
cd "$HOME\Projects"
```

Step 3 — Clone the repository

```
git clone https://github.com/Vikashraise/calculator.git
cd calculator
```

Git creates a local copy of the repository, downloads its history, and configures the remote named origin.

Step 4 — Inspect the repository

```
git status
git remote -v
git branch -a
git log --oneline -10
```

• **git status** — current branch and whether you have local changes.

• **git remote -v** — remote names and their URLs.

• **git branch -a** — local branches and known remote-tracking branches.

• **git log --oneline -10** — the latest ten commits.

Step 5 — Understand the code before editing

The repository shown in class contains **backend/**, **frontend/**, **README.md**, **.gitignore**, and **test_api.py**. Explore the actual files to confirm the architecture.

```
Act as a senior developer joining this project.
Do not create, edit, or delete any files.

1. Explain the folder structure and application architecture.
2. Identify the backend and frontend entry points.
3. Explain how the frontend communicates with the backend.
4. Locate the existing calculation functions.
5. Inspect test_api.py and explain how to run tests.
6. Recommend where a percentage feature belongs.
7. Explain how to run the application locally.

Use actual file paths and code references. Do not guess.
```

Compare the response with the repository files before relying on it.

3. Create a feature branch

A feature branch lets you develop a change without working directly on **main**.

```
git switch -c feature/percentage-calculator
git branch
git status
```

The asterisk (*) in the branch list marks the branch you are currently using.

If several students share one repository, use a unique branch name for each person, such as `feature/percentage-student01`.

4. Implement a small feature

Feature request: add a percentage calculation. For example, **20% of 500 should return 100**.

```
Implement a Percentage Calculator feature in this repository.

- Inspect the existing code before editing.
- Support calculating X percent of Y (20% of 500 = 100).
- Follow the existing architecture and coding conventions.
- Add input validation and automated tests.
- Avoid unnecessary dependencies and unrelated changes.
- Run relevant tests and report the actual results.
- Summarize changed files and manual verification steps.

Do not run Git commands, commit, push, or merge.
I will manage the Git workflow manually.
```

Review the generated code and test it before committing. Check that no secrets, local environment files, or unrelated changes were added.

5. Review, stage, and commit

Step 1 — Review changes

```
git status
git diff
```

git diff shows changes that have not been staged.

Step 2 — Stage the intended files

```
git add .
git status
git diff --cached
```

git add selects changes for the next commit. **git diff --cached** shows what is staged. If unrelated files changed, stage only the files that belong to your feature.

Step 3 — Commit

```
git commit -m "Add percentage calculator feature"
git log --oneline -5
```

A commit saves the staged changes in local Git history. Use a message that describes what changed.

6. Push your branch to GitHub

```
git push -u origin feature/percentage-calculator
```

This uploads your branch to the remote named **origin** and sets upstream tracking. For later pushes from the same branch, you can normally run:

```
git push
```

If GitHub reports that you do not have permission, confirm your repository access or push to an authorized fork.

7. Open a pull request

1. Open the calculator repository on GitHub.
2. Select **Pull requests**, then **New pull request**.
3. Choose **main** as the base branch and your feature branch as the compare branch.
4. Review the diff and write a clear title and summary.
5. Create the pull request and respond to review comments or required checks.

A pull request lets the team review your work before it is merged into main.

8. Merge conflict practice

A merge conflict can happen when two branches change the same lines differently. Use a practice repository or temporary branches for this exercise.

Step 1 — Add a line on main

```
git switch main
# Edit README.md and add:
# Calculator project status: Stable
git add README.md
git commit -m "Update calculator project status"
```

Step 2 — Change the same line on your feature branch

```
git switch feature/percentage-calculator
# Change that same line to:
# Calculator project status: Feature development
git add README.md
git commit -m "Update project status on feature branch"
```

Step 3 — Merge main into the feature branch

```
git merge main
```

If the same line was changed differently, Git should report a conflict. Open README.md and find markers similar to these:

```
<<<<<<< HEAD
Calculator project status: Feature development
=======
Calculator project status: Stable
>>>>>>> main
```

Step 4 — Resolve the conflict

Choose the correct final wording and delete the conflict markers. For example:

```
Calculator project status: Stable release with new features in development
```

Then stage and commit the resolved file:

```
git add README.md
git status
git commit -m "Resolve README merge conflict"
```

Push the updated branch:

```
git push
```

Cancel an unfinished merge

```
git merge --abort
```

Use this only if a merge is still in progress and you want to cancel it.

9. Quick command reference

Command	Purpose
`git clone URL`	Clone a repository
`git status`	Check branch and working-tree state
`git remote -v`	Show remote URLs
`git branch -a`	List local and known remote branches
`git switch -c NAME`	Create and switch to a branch
`git switch main`	Switch to main
`git diff`	View unstaged changes
`git add FILE`	Stage a selected file
`git add .`	Stage changes under the current directory
`git diff --cached`	Review staged changes
`git commit -m "MESSAGE"`	Save staged changes locally
`git push -u origin BRANCH`	Push branch and set upstream
`git fetch origin`	Download remote updates without integrating
`git pull`	Fetch and integrate upstream updates
`git merge main`	Merge main into the current branch
`git merge --abort`	Cancel an unfinished merge
`git log --oneline -10`	View recent commits
`git branch -vv`	View branch and upstream tracking

10. Common issues

Authentication or access denied

Confirm you have permission to access the private repository and are signed in through a supported GitHub credential method.

Push says there is no upstream

Check the branch with `git branch -vv`. For the first push of a new branch, use `git push -u origin BRANCH_NAME`.

Changes are on the wrong branch

Pause and inspect `git status` and `git branch` before taking action. Avoid resetting or deleting work until you know whether it is committed.

git diff shows nothing

The changes may already be staged, untracked, or ignored. Check `git status` and `git diff --cached`.

Conflict markers remain

Do not commit until all conflict markers have been removed, the final content is correct, and relevant tests pass.

Your completion checklist

- ■ I can explain Git, GitHub, clone, remote, branch, commit, push, and pull request.
- ■ I cloned the calculator repository and inspected its history.
- ■ I created and switched to a feature branch.
- ■ I reviewed the code and tests before committing.
- ■ I committed my changes with a clear message.
- ■ I pushed my feature branch and opened a pull request.
- ■ I resolved a merge conflict and understand why it happened.